**UNIVERSIDAD TECNOLÓGICA NACIONAL**
Tecnicatura Universitaria en Programación

# TRABAJO PRÁCTICO - UNIDAD 5

Administración y seguridad de Food Store

Proyecto integrador: Food Store
Roles · Auditoría · Incidentes · Protección de datos

Materia: Base de Datos II
Carrera: Tecnicatura Universitaria en Programación (UTN)

Autores:
Lemus Nahuel
Anglat Juan
Elias Juan Cruz

Año 2026

# 1. Presentación y diseño de roles

Food Store: administración y seguridad
Bases de Datos II - Tecnicatura Universitaria en Programación
Resolución técnica y evidencias del laboratorio - 20/10/2026

## Entorno y alcance

Se trabajó en PostgreSQL 18.6, proyecto postgrefacu de Railway, sobre la copia foodstore. La fuente railway y el contenido de las cinco tablas de negocio se verificaron sin cambios. Se preservó el esquema previo con detalle_pedido y cliente.mail.

La tabla usuario y las funciones de autenticación no estaban conservadas: se reconstruyeron con 18 identidades totalmente sintéticas. No representan cuentas históricas de los 5.000 clientes. El usuario instaló pgcrypto en foodstore; las extensiones pertenecen a cada base.

Estado de la entrega: la implementación y las pruebas técnicas están realizadas. La especificación fue redactada con Codex; Kiro no estuvo disponible y OpenCode falló con HTTP 400. Las intervenciones válidas de esas herramientas y la revisión humana exigidas por la cátedra siguen pendientes. No se presentan como realizadas.

## Parte A. Diseño y mínimo privilegio

| Rol | Operación habilitada y justificación |
|---|---|
| rol_app_lectura | Catálogo, clientes y ventas; columnas no secretas de usuario para la aplicación. |
| rol_app_escritura | INSERT/UPDATE de productos, pedidos y detalles; referencias y auditoría de stock. Sin DELETE. |
| rol_soporte | Diagnóstico de stock/ventas; id, nombre y apellido de cliente. Sin correo ni celular. |
| rol_reportes | Catálogo/ventas e id, rol y fecha_alta de usuario, sin hashes. |
| app_web | LOGIN que hereda lectura/escritura. Sin facultades administrativas. |
| admin_datos | LOGIN con soporte/reportes, UPDATE de rol/activo y reseteo controlado de contraseña. |
| foodstore_auth_owner | NOLOGIN propietario de funciones; acceso acotado a usuario y criptografía. |

app_web y admin_datos son NOSUPERUSER, NOCREATEDB, NOCREATEROLE, NOREPLICATION y NOBYPASSRLS. Los logins quedan sin contraseña permanente. Los roles del motor son globales al clúster, aunque sus privilegios sobre objetos se otorgan en foodstore.

# Parte A. Implementación y verificación

## Permiso de columna y objetos futuros

```
GRANT SELECT (id, nombre, apellido) ON cliente TO rol_soporte;
GRANT SELECT (id, rol, fecha_alta) ON usuario TO rol_reportes;
GRANT UPDATE (rol, activo) ON usuario TO admin_datos;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  REVOKE ALL ON TABLES FROM PUBLIC;
ALTER DEFAULT PRIVILEGES REVOKE EXECUTE ON FUNCTIONS FROM PUBLIC;
```

Se revocaron los permisos de PUBLIC sobre esquema, tablas, secuencias y funciones. Las tablas futuras quedan cerradas y se habilitan expresamente cuando se conoce su contenido. El REVOKE de EXECUTE es global: revocarlo solo por esquema no elimina el permiso público global de creación de funciones. Los defaults corresponden al usuario creador; otro propietario debe configurar su propia política.

## Autenticación reconstruida

usuario incluye id, nombre, mail, rol, fecha_alta, activo y password_hash. Los hashes se generan con crypt y gen_salt de pgcrypto, usando bcrypt con costo 10 y sal aleatoria. Las claves de prueba se generan en memoria y no se guardan en scripts.

fn_autenticar(text,text) devuelve el id cuando coinciden correo, contraseña y estado activo; devuelve NULL si falla. fn_resetear_password(bigint,text) solo puede ejecutarla admin_datos y exige al menos 16 caracteres. Ambas son SECURITY DEFINER, con propietario NOLOGIN, nombres de objetos calificados y search_path fijo a pg_catalog,pg_temp. Nunca devuelven el hash.

## Verificación efectiva

| Prueba | Resultado observado |
|---|---|
| Lectura de productos como app_web | Permitida; 1.000 productos. |
| UPDATE de stock como app_web | Permitido y revertido mediante ROLLBACK. |
| Lectura de password_hash como app_web | Denegada: SQLSTATE 42501. |
| DELETE de pedidos como app_web | Denegado: SQLSTATE 42501. |
| Reseteo de contraseña como app_web | Denegado: SQLSTATE 42501. |
| Soporte: id/nombre/apellido de cliente | Permitido. |
| Soporte: correo de cliente | Denegado: SQLSTATE 42501. |
| Tabla y función futuras de prueba | Sin SELECT/EXECUTE para app_web; creación revertida. |

Se adjunta la salida de \du y de information_schema.role_table_grants/role_column_grants en docs/verificacion_roles.txt. El borrador concedía CREATEDB/CREATEROLE a admin_datos y lectura de tablas futuras: se corrigió por ser excesivo. No se atribuye esa propuesta a una respuesta de OpenCode no obtenida.

# Parte B. Auditoría

| Parámetro | Valor verificado |
|---|---|
| log_connections | on |
| log_disconnections | on |
| log_statement | mod global; all solo en sesión de lectura de prueba |
| log_line_prefix | Fecha, PID, usuario, base, aplicación y origen |

La configuración global había sido aplicada por el usuario y se verificó mediante pg_settings; Codex no ejecutó ALTER SYSTEM. La sesión de prueba efectuó una lectura de cliente, inserción de categoría y actualización de stock, seguidas de ROLLBACK. Además se rechazó un intento de conexión SQL con un rol inexistente.

## Fragmento real de log, reducido y anonimizado

```
2026-10-20 21:03:03 UTC | statement: SELECT count(*) clientes FROM cliente;

2026-10-20 21:03:04 UTC | statement: INSERT INTO categoria(nombre,descripcion) VALUES
('U5_AUDITORIA_SINTETICA','Prueba con rollback');

2026-10-20 21:03:04 UTC | statement: UPDATE producto SET stock=stock WHERE id=(SELECT min(id) FROM producto);

2026-10-20 21:03:06 UTC | password authentication failed for user "u5_login_inexistente"
```

## Interpretación y límites

Quién: usuario del motor y actor_motor en eventos de autenticación. Qué: consulta o evento registrado. Cuándo: marca temporal UTC. Desde dónde: dirección sustituida por IP_SINTETICA en la evidencia. El prefijo puede indicar postgres porque la conexión original es administrativa; SET SESSION AUTHORIZATION cambia la identidad efectiva para probar permisos. No equivale a probar un login permanente de app_web.

log_statement=mod registra modificaciones, pero no SELECT; por eso se habilitó all únicamente para la sesión correspondiente. El log de una sentencia no demuestra cuántas filas se recibieron. pgaudit puede ampliar la auditoría de sentencias/objetos; la captura de valores anteriores y nuevos fila por fila requiere una estrategia adicional, por ejemplo triggers específicos.

## Sensibilidad de los logs y corrección aplicada

El log es un activo sensible: puede revelar consultas, identidades y secretos. Solo administradores y auditores autorizados deberían leerlo; los actores auditados no deberían modificarlo. Se requieren permisos restrictivos, retención acotada y almacenamiento protegido.

En el primer ensayo, RAISE LOG adjuntó la sentencia con parámetros sintéticos aun con log_statement=none. Se corrigió usando log_min_error_statement=panic durante las llamadas sensibles, junto con log_statement=none y longitudes de parámetros en cero. La repetición produjo cuatro eventos de autenticación y cero sentencias sensibles adjuntas. El log exportado también redacta parámetros y origen.

## Parte C. Simulacro de incidente

El escenario se ejecutó exclusivamente en el laboratorio propio. laboratorio_u5.py contiene la orquestación ejecutable y sus aserciones; sql/simulacro_incidente.sql documenta el orden de llamadas sin guardar claves.

| Hora UTC del cliente | Evento | Resultado |
|---|---|---|
| 21:02:52.695 | Autenticación fallida | Intento 1: NULL |
| 21:02:52.979 | Autenticación fallida | Intento 2: NULL |
| 21:02:53.821 | Autenticación fallida | Intento 3: NULL |
| 21:02:54.100 | Autenticación exitosa | id = 1 |
| 21:02:55.047 | Lectura masiva de usuario | 18 filas recibidas |
| 21:02:55.823 | Escalamiento rechazado | SQLSTATE 42501 |

```
SET SESSION AUTHORIZATION app_web;
-- Tres fallos y luego un éxito de fn_autenticar, con parámetros en memoria.
SELECT id,nombre,mail,rol,fecha_alta,activo FROM usuario;
GRANT admin_datos TO app_web; -- Rechazado: 42501
```

La lectura recibió realmente todas las filas sintéticas; no se reemplazó por COUNT. Se excluyó password_hash. El GRANT se ensayó en una transacción y el orquestador aborta si llega a permitirlo. Al finalizar, pg_has_role confirmó que app_web no es miembro de admin_datos.

### Reconstrucción asistida y contraste

La reconstrucción con Codex identifica fallos seguidos de éxito, lectura de usuarios e intento denegado de ampliar privilegios. Coincide con lo ejecutado y con los eventos del servidor. El conteo de 18 filas se acredita en el registro del cliente; no puede deducirse solo del texto de SELECT en el log.

El registro no prueba robo de contraseñas, identidad de una persona, automatización de fuerza bruta ni exfiltración fuera del cliente de laboratorio. Tampoco permite bloquear una IP real a partir de IP_SINTETICA. La respuesta válida de OpenCode y el contraste humano exigidos por la consigna siguen pendientes: la invocación de esa herramienta falló antes de generar su análisis.

### Propuesta de contención para decisión del estudiante

Ante un incidente real se propone suspender temporalmente la cuenta afectada, preservar evidencia y revisar membresías y origen. Si aparecen indicios de compromiso, rotar credenciales y limitar intentos. No se ejecutó una contención real ni se automatizó una decisión sugerida por IA; la decisión final y su justificación corresponden al estudiante o equipo.

# Parte D. Protección de datos y equivalencia

Se construyó usuario_anon con nombre y correo sintéticos deterministas, conservando id, rol y mes de alta en UTC. Los datos de origen ya son identidades de laboratorio; la equivalencia se mide sobre su rol y fecha_alta, sin inventar un mes común para todos los clientes.

```
INSERT INTO usuario_anon
SELECT id,rol,date_trunc('month',fecha_alta AT TIME ZONE 'UTC')::date,
       'Usuario_'||lpad(id::text,6,'0'),'anon_'||id||'@example.invalid'
FROM usuario;

-- Agregación de la fuente
SELECT rol,date_trunc('month',fecha_alta AT TIME ZONE 'UTC')::date AS mes_alta,
       count(*) AS cantidad
FROM usuario GROUP BY 1,2;

-- Agregación de la tabla seudonimizada
SELECT rol,mes_alta,count(*) AS cantidad
FROM usuario_anon GROUP BY 1,2;
```

## Resultados lado a lado

| Rol | Mes de alta | usuario | usuario_anon |
|---|---|---|---|
| cliente | 2026-01-01 | 3 | 3 |
| cliente | 2026-04-01 | 3 | 3 |
| operador | 2026-02-01 | 3 | 3 |
| operador | 2026-05-01 | 3 | 3 |
| soporte | 2026-03-01 | 3 | 3 |
| soporte | 2026-06-01 | 3 | 3 |
| TOTAL | | 18 | 18 |

Se compararon los conjuntos en ambas direcciones con EXCEPT: 0 diferencias. El script incluye una aserción automática; la salida original de ambas consultas está en docs/verificacion_equivalencia.txt.

## Alcance de la técnica

Conservar un id estable y grupos pequeños mantiene posibilidades de vinculación: es seudonimización determinista, no garantía de anonimato irreversible. Cambiar nombres no basta si quedan identificadores o combinaciones reconocibles.

No deben enviarse a un asistente nombres, correos, teléfonos, direcciones, contraseñas, hashes ni compras vinculables sin tratamiento previo y autorización. Para este ejercicio se comparten resultados agregados y logs redactados de un conjunto completamente sintético.

# Parte E. Bitácora y trazabilidad

| Intervención registrada | Respuesta y verificación |
|---|---|
| A. Resolver U5/U6 con Railway | Codex reconstruyó roles y funciones. Se probaron permisos efectivos positivos y negativos. El usuario instaló pgcrypto. Revisión humana pendiente. |
| B. Verificar auditoría y capturar eventos | Se verificó configuración, lectura, INSERT/UPDATE con rollback y login fallido. El defecto de parámetros en RAISE LOG se observó, corrigió y volvió a probar. |
| C. Simular acceso y escalamiento | Tres fallos, éxito, lectura de 18 filas y GRANT rechazado 42501. Se verificó ausencia de membresía administrativa. Contención humana pendiente. |
| D. Comparar agrupamientos | Mismo rol/mes UTC en ambas consultas. EXCEPT bidireccional devolvió cero diferencias. |
| Revisión solicitada a OpenCode | Se envió el SQL para auditar mínimo privilegio, SECURITY DEFINER y defaults sin cambios. Falló HTTP 400 por esquema MCP inválido; el error completo quedó guardado. No produjo una auditoría válida. |

## Error significativo y corrección

El borrador otorgaba CREATEDB/CREATEROLE a admin_datos y lectura automática de tablas futuras. Se sustituyó por privilegios puntuales y objetos nuevos cerrados. Se comprobó además que el EXECUTE público de funciones debía revocarse globalmente, creando una función de prueba y verificando que app_web no podía ejecutarla. La transacción de prueba se revirtió.

El hallazgo de parámetros en RAISE LOG fue otro error real corregido. Estas correcciones técnicas fueron hechas y verificadas con Codex; no se inventa una recomendación insegura de OpenCode ni una validación humana no realizada para satisfacer la bitácora.

## Historial Git local

```
3b1c8f1 U6: presentar resolucion con caratula y formato de entregas anteriores
28f7cf4 U5: agregar resolucion en PDF con evidencias y SQL
cca1e00 U6: verificar respaldo restauracion y RPO RTO con informe
49de68b U5 E: registrar asistencia real y requisitos de proceso pendientes
5141cf0 U5 D: comprobar equivalencia por rol y mes de usuarios sinteticos
c7911cd U5 C: probar incidente y rechazo efectivo del escalamiento
```

El historial conserva avances por partes en la rama tp-u5-u6-railway. Los commits son locales; este informe no afirma publicación o entrega al aula. BITACORA.md registra prompts, respuestas resumidas, correcciones y pendientes.

## Pendientes de proceso académico

Completar requirements/design con Kiro, obtener auditoría y reconstrucción válidas de OpenCode y conservar su respuesta completa. El estudiante debe contrastar esa respuesta con la ejecución, validar la resolución y decidir la contención. Estos requisitos se distinguen expresamente de los resultados técnicos ya verificados.

# Anexo. SQL de la resolución

Código aplicado en foodstore. roles.sql es una instalación inicial: se detiene ante roles existentes y no debe repetirse como si fuera una migración idempotente. Las pruebas se ejecutan con laboratorio_u5.py y el cliente compartido laboratorio/railway_client.py. Las credenciales se obtienen de la sesión de Railway o de un entorno efímero.

## roles.sql

```
-- Parte A. Primera instalacion en foodstore; abortar ante roles existentes.
BEGIN;
DO $$ BEGIN
 IF current_database()<>'foodstore' THEN RAISE EXCEPTION 'Usar foodstore'; END IF;
 IF NOT EXISTS(SELECT FROM pg_extension WHERE extname='pgcrypto') THEN RAISE EXCEPTION 'Instalar pgcrypto
manualmente'; END IF;
END $$;
CREATE ROLE rol_app_lectura NOLOGIN;
CREATE ROLE rol_app_escritura NOLOGIN;
CREATE ROLE rol_soporte NOLOGIN;
CREATE ROLE rol_reportes NOLOGIN;
CREATE ROLE app_web LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS;
CREATE ROLE admin_datos LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS;
CREATE ROLE foodstore_auth_owner NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE;
-- Sin passwords permanentes; validacion mediante SET SESSION AUTHORIZATION.
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO
rol_app_lectura,rol_app_escritura,rol_soporte,rol_reportes,foodstore_auth_owner;
GRANT SELECT ON categoria,producto,cliente,pedido,detalle_pedido TO rol_app_lectura;
GRANT SELECT ON categoria,producto,pedido,detalle_pedido TO rol_reportes,rol_soporte;
GRANT SELECT(id,nombre,apellido) ON cliente TO rol_soporte;
GRANT SELECT ON auditoria_stock TO rol_soporte;
GRANT SELECT,INSERT,UPDATE ON producto,pedido,detalle_pedido TO rol_app_escritura;
GRANT SELECT ON categoria,cliente TO rol_app_escritura;
GRANT USAGE ON SEQUENCE producto_id_seq,pedido_id_seq,detalle_pedido_id_seq,auditoria_stock_id_seq TO
rol_app_escritura;
GRANT INSERT ON auditoria_stock TO rol_app_escritura;
GRANT rol_app_lectura,rol_app_escritura TO app_web;
GRANT rol_soporte,rol_reportes TO admin_datos;
ALTER DEFAULT PRIVILEGES IN SCHEMA public REVOKE ALL ON TABLES FROM PUBLIC;
ALTER DEFAULT PRIVILEGES IN SCHEMA public REVOKE ALL ON SEQUENCES FROM PUBLIC;
-- Global: revocar solo en esquema no elimina el EXECUTE publico global.
ALTER DEFAULT PRIVILEGES REVOKE EXECUTE ON FUNCTIONS FROM PUBLIC;
CREATE TABLE usuario(
 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,nombre text NOT NULL,
 mail text UNIQUE NOT NULL,rol text NOT NULL CHECK(rol IN ('cliente','operador','soporte')),
 fecha_alta timestamptz NOT NULL,activo boolean NOT NULL DEFAULT true,password_hash text NOT NULL);
-- Identidades enteramente sinteticas, independientes de cliente.
INSERT INTO usuario(nombre,mail,rol,fecha_alta,password_hash)
SELECT 'Persona sintetica '||n,'persona'||n||'@example.invalid',
 (ARRAY['cliente','operador','soporte'])[1+(n%3)],
 timestamptz '2026-01-01 00:00:00+00'+(n%6)*interval '1 month',
 public.crypt(encode(gen_random_bytes(24),'hex'),public.gen_salt('bf',10))
FROM generate_series(1,18) n;
GRANT SELECT(id,nombre,mail,rol,fecha_alta,activo) ON usuario TO rol_app_lectura;
GRANT SELECT(id,rol,fecha_alta) ON usuario TO rol_reportes;
GRANT SELECT(id,nombre,rol,activo) ON usuario TO rol_soporte;
GRANT UPDATE(rol,activo) ON usuario TO admin_datos;
GRANT SELECT,UPDATE(password_hash) ON usuario TO foodstore_auth_owner;
GRANT EXECUTE ON FUNCTION public.crypt(text,text),public.gen_salt(text,integer) TO foodstore_auth_owner;
CREATE FUNCTION fn_autenticar(p_mail text,p_password text) RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER SET search_path=pg_catalog,pg_temp AS $$
DECLARE found_id bigint;
BEGIN
 SELECT u.id INTO found_id FROM public.usuario u WHERE u.mail=p_mail AND u.activo
 AND u.password_hash=public.crypt(p_password,u.password_hash);
 RAISE LOG 'U5_AUTH resultado=% actor_motor=%',CASE WHEN found_id IS NULL THEN 'fallido' ELSE 'exitoso'
END,session_user;
 RETURN found_id;
END $$;
CREATE FUNCTION fn_resetear_password(p_id bigint,p_password text) RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path=pg_catalog,pg_temp AS $$
BEGIN
 IF length(p_password)<16 THEN RAISE EXCEPTION 'Minimo 16 caracteres'; END IF;
 UPDATE public.usuario SET password_hash=public.crypt(p_password,public.gen_salt('bf',10)) WHERE id=p_id;
 IF NOT FOUND THEN RAISE EXCEPTION 'Usuario inexistente'; END IF;
 RAISE LOG 'U5_RESET actor_motor=%',session_user;
END $$;
GRANT CREATE ON SCHEMA public TO foodstore_auth_owner;
ALTER FUNCTION fn_autenticar(text,text) OWNER TO foodstore_auth_owner;
ALTER FUNCTION fn_resetear_password(bigint,text) OWNER TO foodstore_auth_owner;
REVOKE CREATE ON SCHEMA public FROM foodstore_auth_owner;
REVOKE ALL ON FUNCTION fn_autenticar(text,text),fn_resetear_password(bigint,text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION fn_autenticar(text,text) TO app_web;
GRANT EXECUTE ON FUNCTION fn_resetear_password(bigint,text) TO admin_datos;
COMMIT;
\du
SELECT grantee,table_name,privilege_type FROM information_schema.role_table_grants
WHERE grantee IN
('rol_app_lectura','rol_app_escritura','rol_soporte','rol_reportes','admin_datos','foodstore_auth_owner')
ORDER BY 1,2,3;
SELECT grantee,table_name,column_name,privilege_type FROM information_schema.role_column_grants
WHERE table_name IN ('usuario','cliente') AND grantee LIKE 'rol_%' ORDER BY 1,2,3,4;
```

## auditoria_test.sql

```
-- Parte B. Configuracion global aplicada previamente por el usuario.
SELECT name,setting,pending_restart FROM pg_settings WHERE name IN
('log_connections','log_disconnections','log_statement','log_line_prefix');
SET application_name='foodstore_u5_auditoria';
SET log_statement='all';
SELECT count(*) clientes FROM cliente;
BEGIN;
INSERT INTO categoria(nombre,descripcion) VALUES ('U5_AUDITORIA_SINTETICA','Prueba con rollback');
UPDATE producto SET stock=stock WHERE id=(SELECT min(id) FROM producto);
ROLLBACK;
-- Intento de conexion SQL fallida: laboratorio_u5.py.
```

## simulacro_incidente.sql

```
-- Ejecucion parametrizada en laboratorio_u5.py (sin secretos en disco).
-- SET SESSION AUTHORIZATION app_web;
-- SELECT fn_autenticar($1,$2); tres veces con password incorrecto: NULL.
-- SELECT fn_autenticar($1,$2); password correcto: id no nulo.
-- SELECT id,nombre,mail,rol,fecha_alta,activo FROM usuario; lectura de todas las filas.
-- GRANT admin_datos TO app_web; debe fallar SQLSTATE 42501.
-- No se lee password_hash; no se imprimen filas personales.
-- Si GRANT tiene exito se hace ROLLBACK y se aborta el ensayo.
```

# Anexo. Anonimización y evidencia

## usuario_anon.sql

```
-- Parte D. No se inventa un mes unico para todos los clientes.
BEGIN;
CREATE TABLE IF NOT EXISTS usuario_anon(id bigint PRIMARY KEY,rol text NOT NULL,mes_alta date NOT
NULL,nombre_sintetico text NOT NULL,mail_sintetico text UNIQUE NOT NULL);
TRUNCATE usuario_anon;
INSERT INTO usuario_anon SELECT id,rol,date_trunc('month',fecha_alta AT TIME ZONE
'UTC')::date,'Usuario_'||lpad(id::text,6,'0'),'anon_'||id||'@example.invalid' FROM usuario;
COMMIT;
SELECT 'usuario' fuente,rol,date_trunc('month',fecha_alta AT TIME ZONE 'UTC')::date mes_alta,count(*) cantidad
 FROM usuario GROUP BY 2,3
UNION ALL SELECT 'usuario_anon',rol,mes_alta,count(*) FROM usuario_anon GROUP BY 2,3 ORDER BY 2,3,1;
WITH a AS(SELECT rol,date_trunc('month',fecha_alta AT TIME ZONE 'UTC')::date mes_alta,count(*) cantidad FROM
usuario GROUP BY 1,2),b AS(SELECT rol,mes_alta,count(*) cantidad FROM usuario_anon GROUP BY 1,2)
SELECT count(*) diferencias FROM((SELECT * FROM a EXCEPT SELECT * FROM b) UNION ALL (SELECT * FROM b EXCEPT
SELECT * FROM a)) d;
-- Tecnica determinista con id estable: seudonimizacion, no anonimato irreversible.
```

## Archivos de evidencia incluidos en el paquete

docs/verificacion_roles.txt; salida_auditoria.txt; log_auditoria_prueba.txt; log_simulacro_anonimizado.txt; pruebas_reales.json; verificacion_final.json; verificacion_equivalencia.txt; opencode_auditoria.jsonl. Los logs no incluyen contraseñas ni filas personales; las direcciones están sustituidas.

Las cinco tablas de negocio se compararon con el resumen de contenido anterior: categoria 20, producto 1.000, cliente 5.000, pedido 20.000 y detalle_pedido 20.000. Tanto railway como foodstore conservaron esos datos. Las nuevas identidades sintéticas se agregaron únicamente a la copia de trabajo.